

Python / FastAPI / LangGraph + OpenAI API

☒ 学習ロードマップ

Phase	テーマ	ファイル
1	環境構築・OpenAI API最小呼び出し	test_api.py
2	FastAPIでチャットエンドポイントを作る	main.py
3	会話履歴を持たせる（マルチターン対応）	main.py（更新版）
5	LangGraphの基礎（State / Node / Edge）	test_langgraph.py
6	ツール呼び出しエージェント	agent.py

Phase 1 : test_api.py OpenAI API 最小呼び出し

概要

OpenAI APIを直接呼び出す最もシンプルなコード。LangGraphやFastAPIを使わずに、まずAPIとの通信を確認する。

コード

[illegible]

コード解説

- ・ `load_dotenv()` → `.env` ファイルを読み込み、`OPENAI_API_KEY` を環境変数として登録する
- ・ `OpenAI()` → APIキーを自動取得してクライアントを作成。引数不要。
- ・ `messages` → 会話の履歴をリストで渡す。role は `system` / `user` / `assistant` の3種類
- ・ `system` → モデルの振る舞い・人格を設定する裏設定（ユーザーには見えない）
- ・ `user` → 人間からの発言
- ・ `choices[0].message.content` → モデルの返答テキストを取り出す
- ・ `response.usage.total_tokens` → 入力+出力のトークン合計（課金単位）

実行方法

```
python test_api.py
```

期待される出力例

[illegible]

Phase 2 : main.py (初期版) FastAPIでチャットAPIを作る

概要

test_api.pyの処理をFastAPIのWebエンドポイントとしてラップする。ブラウザやクライアントからHTTPリクエストでAIと会話できるようになる。

コード

```

from dotenv import load_dotenv
from openai import OpenAI
from fastapi import FastAPI
from pydantic import BaseModel

load_dotenv()
client = OpenAI()
app = FastAPI()

class ChatRequest(BaseModel):
    message: str # [REDACTED]

@app.post("/chat")
def chat(request: ChatRequest):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "[REDACTED]"},
            {"role": "user", "content": request.message}
        ]
    )
    return {
        "reply": response.choices[0].message.content,
        "tokens": response.usage.total_tokens
    }

```

コード解説

- ・ FastAPI() → Webアプリのインスタンスを作成
- ・ BaseModel → リクエストボディの型をPydanticで定義。バリデーションが自動で行われる
- ・ @app.post('/chat') → POSTリクエストを受け取るエンドポイントを定義
- ・ request.message → リクエストボディのmessageフィールドを取得

実行方法

```
uvicorn main:app --reload
```

起動後、ブラウザで以下を開く：

```
http://127.0.0.1:8000/docs
```

- ・ Swagger UI（自動生成のテスト画面）が開く
- ・ /chat を展開 → Try it out → messageに文章を入力 → Execute

Phase 3 : main.py (更新版) 会話履歴を持たせる

概要

Phase 2のコードは毎回新しい会話を作っていた（前回の発言を覚えていない）。`session_id`ごとに会話履歴をメモリ上に保存することで、マルチターン会話を実現する。

コード

[illegible]

コード解説

- ・ conversation_history → Pythonの辞書。キー=session_id、値=messages配列
- ・ session_id → 同じIDで送ると「同じ会話の続き」として処理される
- ・ history.append() → ユーザーの発言と返答を両方履歴に追加していく
- ・ history_length → ターンが増えるたびに2ずつ増える (user + assistant で1往復=2件)
- ・ 注意：サーバーを再起動すると履歴は消える。本番ではDBへの永続化が必要

動作確認 (Swagger UIで2回送信)

```
1■■: {"session_id": "test1", "message": "■■■■■■■■■■"}
2■■: {"session_id": "test1", "message": "■■■■■■■■■■"}
→ ■■■■■■■■■■■■■■■■
```

Phase 5 : test_langgraph.py LangGraphの基礎

概要

Phase 3で手動管理していた「状態（会話履歴）と処理の流れ」をLangGraphのフレームワークで形式化する。State / Node / Edgeという3つの概念を理解することが目標。

LangGraphの概念	Phase 3での対応物
State（状態）	conversation_history の中身
Node（ノード）	/chat 関数の中の処理
Edge（エッジ）	次にどの処理に進むかの道筋

コード

```
from typing import TypedDict
from dotenv import load_dotenv
from openai import OpenAI
from langgraph.graph import StateGraph, START, END

load_dotenv()
client = OpenAI()

# ① State ████████████████████████████████
class ChatState(TypedDict):
    messages: list[dict]

# ② Node ████████████████████████████████████████████
def call_model(state: ChatState) -> ChatState:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=state["messages"]
    )
    reply = response.choices[0].message.content
    state["messages"].append({"role": "assistant", "content": reply})
    return state

# ③ ██████████
graph = StateGraph(ChatState)
graph.add_node("call_model", call_model) # ██████████
graph.add_edge(START, "call_model") # ███ → call_model
graph.add_edge("call_model", END) # call_model → ███
app = graph.compile() # ████████████████████

# ④ ███
initial_state = {
    "messages": [
        {"role": "system", "content": "████████████████████"},
        {"role": "user", "content": "████████████████████"}
    ]
}

result = app.invoke(initial_state)
```

```
print(result["messages"][-1]["content"])    # [REDACTED]AI[REDACTED]
```

コード解説

- ・ TypedDict → Stateの型を定義するためのPython標準機能
- ・ StateGraph(ChatState) → このグラフが扱うデータ型を宣言してグラフを作成
- ・ add_node() → 処理関数をノードとして登録する
- ・ add_edge(START, 'call_model') → 開始時に最初に行うノードを指定
- ・ add_edge('call_model', END) → call_model実行後は終了
- ・ graph.compile() → グラフ定義を実行可能なappオブジェクトに変換
- ・ app.invoke(initial_state) → 初期状態を渡して実行。最終状態が返る
- ・ result['messages'][-1] → messagesリストの最後の要素 (AIの最新返答)

実行方法

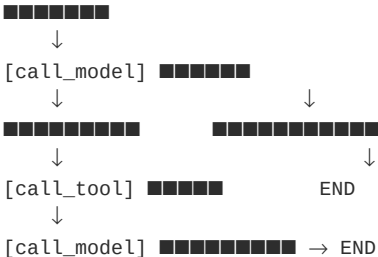
```
python test_langgraph.py
```

Phase 6 : agent.py ツール呼び出しエージェント

概要

モデルが「ツールを呼ぶかどうか」を自分で判断し、必要なら実際に関数を実行して結果を使って最終回答を作る。
これがAIエージェントの基本形（ReActパターン）。

処理フロー



コード

```
import json
from typing import TypedDict
from dotenv import load_dotenv
from openai import OpenAI
from langgraph.graph import StateGraph, START, END

load_dotenv()
client = OpenAI()

# ===== ██████████ =====
def get_weather(city: str) -> str:
    weather_data = {"████": "██████28██", "██": "██████25██"}
    return weather_data.get(city, f"{city}████████████████████")

def calculate(expression: str) -> str:
    try:
        return f"████: {eval(expression)}"
    except Exception as e:
        return f"██████: {str(e)}"
```

```

# ===== OpenAI =====
tools = [
    {"type": "function", "function": {
        "name": "get_weather",
        "description": " ",
        "parameters": {"type": "object",
            "properties": {"city": {"type": "string", "description": " "}},
            "required": ["city"]}
    }},
    {"type": "function", "function": {
        "name": "calculate",
        "description": " ",
        "parameters": {"type": "object",
            "properties": {"expression": {"type": "string", "description": " "}},
            "required": ["expression"]}
    }}
]
tool_map = {"get_weather": get_weather, "calculate": calculate}

# ===== State / Node / Edge =====
class AgentState(TypedDict):
    messages: list[dict]

def call_model(state: AgentState) -> AgentState:
    response = client.chat.completions.create(
        model="gpt-4o-mini", messages=state["messages"],
        tools=tools, tool_choice="auto"
    )
    message = response.choices[0].message
    state["messages"].append({
        "role": "assistant", "content": message.content,
        "tool_calls": [{ "id": tc.id, "type": "function",
            "function": { "name": tc.function.name,
                "arguments": tc.function.arguments}}
            for tc in (message.tool_calls or []) or None
    })
    return state

def call_tool(state: AgentState) -> AgentState:
    for tool_call in (state["messages"][-1].get("tool_calls") or []):
        func_name = tool_call["function"]["name"]
        func_args = json.loads(tool_call["function"]["arguments"])
        result = tool_map[func_name](**func_args)
        state["messages"].append({"role": "tool",
            "tool_call_id": tool_call["id"], "content": result})
    return state

def should_continue(state: AgentState) -> str:
    # 
    return "call_tool" if state["messages"][-1].get("tool_calls") else END

# ===== 
graph = StateGraph(AgentState)
graph.add_node("call_model", call_model)
graph.add_node("call_tool", call_tool)
graph.add_edge(START, "call_model")
graph.add_conditional_edges("call_model", should_continue,
    {"call_tool": "call_tool", END: END})
graph.add_edge("call_tool", "call_model") # 
app = graph.compile()

# ===== 
def run_agent(user_message: str):
    result = app.invoke({"messages": [
        {"role": "system", "content": " "},

```

```
run_agent("■■■■■■■■■■■■■■■■■■■■")
run_agent("1234 * 5678 ■■■■■■■■■■■■■■■■■■■■")
run_agent("■■■■■■■■■■■■■■■■■■■■") # ■■■■■■■■■■■■■■■■■■■■
```

コード解説

- ・ `tools` → OpenAIにツールの仕様（名前・説明・引数）を辞書形式で教える
- ・ `tool_choices='auto'` → モデルが必要に応じて自動でツールを選択する
- ・ `tool_calls` → モデルがツールを使いたい場合、ここに呼び出し情報が入る
- ・ `tool_map` → ツール名と実際のPython関数を紐付けるdict
- ・ `should_continue()` → `tool_calls`があれば'call_tool'へ、なければENDへの条件分岐
- ・ `add_conditional_edges()` → ノードの出力によって次のノードを動的に決める
- ・ `call_tool` → `call_model` のEdge → 「考える→行動→また考える」ループを実現

実行方法

```
python agent.py
```

期待される出力例

[illegible]

3つ目の「こんにちは」はツールを呼ばずに直接回答します。モデル自身が判断しています。

☒ 全体まとめ LangGraph の3つの核心概念

概念	役割	コードでの対応
State	グラフ全体で共有されるデータ	TypedDictで定義したクラス
Node	状態を受け取り処理して返す関数	add_node()で登録した関数
Edge	次にどのノードに進むかの道筋	add_edge() / add_conditional_edges()

次のステップ (Phase

7) は、FastAPIとLangGraphを統合して、Webエンドポイント経由でエージェントを動かす実践的な構成です。

以上